



Notions de base

Calculer avec Python

Sous Python 3 :

- L'opérateur de division `/` effectue une division réelle
- Pour obtenir une division entière (le résultat est un entier), utiliser `//`

Tester

- **`20.5/3` vs `20.5//3`**
- **`8.7/5`**

Attention : taper le `.` et pas le `,`

Opérateurs

- Opérations mathématiques
*, +, /, //, ** (exponentiation)
- Opérateur modulo : %

Fournit le reste de la division entière

❖ Testez :

10 % 3

10 % 5

Variable

- une variable est un **conteneur utilisé pour stocker un élément de données**
- une variable peut être vue comme **une boîte qui contient une valeur.**
 - Cette boîte est **stockée** sur une étagère dans un entrepôt gigantesque.
 - L'**emplacement** de chaque boîte de cet entrepôt est soigneusement **enregistré**, tout comme votre ordinateur enregistre l'emplacement de votre variable en mémoire.
- Une **donnée placée dans une variable** s'appelle une ***valeur.***
- Chaque variable **a un nom**
 - Le nom d'une variable doit refléter la signification de son contenu

Noms de variables

- Une **séquence de lettres et de chiffres commençant par une lettre**
 - Seules les lettres ordinaires sont autorisées et le caractère `_` (souligné)
- La casse est significative
- Pas utiliser les mots réservés

<code>and</code>	<code>as</code>	<code>assert</code>	<code>break</code>	<code>class</code>	<code>continue</code>	<code>def</code>
<code>del</code>	<code>elif</code>	<code>else</code>	<code>except</code>	<code>False</code>	<code>finally</code>	<code>for</code>
<code>from</code>	<code>global</code>	<code>if</code>	<code>import</code>	<code>in</code>	<code>is</code>	<code>lambda</code>
<code>None</code>	<code>nonlocal</code>	<code>not</code>	<code>or</code>	<code>pass</code>	<code>raise</code>	<code>return</code>
<code>True</code>	<code>try</code>	<code>while</code>	<code>with</code>	<code>yield</code>		

Noms de variable

- utilisez des **noms descriptifs** tout au long de votre code
 - *nombreCar* à la place de *nombre*
- **évit**ez si possible d'**abr**éger ou de raccourcir les mots
 - *chiffreDAffairesAnnuel* est préférable à *chifAfAnn*
- **noms composés**
 - sans espaces ni ponctuation.
 - le premier mot est écrit en minuscules et tous les autres mots commencent par une majuscule.
 - *monBudget*.

Affectations

- Le signe =

n=7; m="Bonjour" ; pi=3.14159

⇒ Les **noms de variables** sont des **références mémorisées** dans une zone de mémoire appelée **espace de noms**

⇒ Les **valeurs** correspondantes sont **situées ailleurs**.

- Afficher la valeur

```
>>> n #ne marche pas sur Spyder
```

```
>>> print(n)
```

Attention : **ne pas oublier les parenthèses avec print**

Affectations multiples

- Assigner une valeur à plusieurs variables simultanément

```
>>> x = y = 7
>>> x
7
>>> y
7
```

- Affectations parallèles

```
>>> a, b = 4, 8.33
>>> a
4
>>> b
8.33
```


Contrôle du flux d'exécution

- La **description structurée d'une série d'action** et de **l'ordre** dans lequel il convient de les effectuer s'appelle un **algorithme**
- Le « **chemin** » suivi par **Python** à travers un programme est appelé un **flux d'exécution**
- Les **constructions qui modifient le programme** sont appelées les **instructions de contrôle de flux**
- Les **structures de contrôle** sont les **groupes d'instructions qui déterminent l'ordre** dans lequel les actions sont effectuées

Instruction *if*

- *if* (si) permet de **tester la validité de la condition.**
- *else* (sinon) permet de programmer une **exécution alternative**
- *elif* (*else if=sinon si*)

```
>>> a = 0
>>> if a > 0 :
...     print("a est positif")
... elif a < 0 :
...     print("a est négatif")
... else:
...     print("a est nul")
... 
```

Opérateurs de comparaison

```
x == y      # x est égal à y
x != y      # x est différent de y
x > y       # x est plus grand que y
x < y       # x est plus petit que y
x >= y      # x est plus grand que, ou égal à y
x <= y      # x est plus petit que, ou égal à y
```

Exemple

```
>>> a = 7
>>> if (a % 2 == 0):
...     print("a est pair")
...     print("parce que le reste de sa division par 2 est nul")
... else:
...     print("a est impair")
... 
```

Instructions imbriquées

```
if embranchement == "vertébrés":           # 1
    if classe == "mammifères":             # 2
        if ordre == "carnivores":         # 3
            if famille == "félins":        # 4
                print("c'est peut-être un chat") # 5
            print("c'est en tous cas un mammifère") # 6
        elif classe == "oiseaux":         # 7
            print("c'est peut-être un canari") # 8
    print("la classification des animaux est complexe") # 9
```

Ne pas tester, juste pour la présentation de la structure d'un script

Ré-affectation

- Ré-affectation : remplacer l'ancienne valeur d'une variable par une nouvelle

```
>>> altitude = 320
>>> print(altitude)
320
>>> altitude = 375
>>> print(altitude)
375
```

- Symbole *égale* utilisé pour une affectation est différent du symbole *égalité*
 - L'égalité est commutative : En mathématique, les écritures : $a=7$ et $7=a$ sont équivalentes alors qu'une instruction `375=altitude` est illégale
 - L'égalité est permanente alors que l'affectation peut être remplacée



Exercice

```
>>> a, b, c, d = 3, 4, 5, 7
```

Comment échanger les valeurs des variables **a** et **b** ?

On veut que **a** contienne la valeur **4** et **b** la valeur **3**.

L'instruction *for*

- Quand **on sait le nombre d'itération**, on utilise généralement une boucle *for*
- L'instruction *for* **se termine par : suivi d'un bloc indenté** qui constitue le corps de la boucle
- On précise après le mot-clé *for*
 - le nom d'une **variable qui prendra successivement toutes les valeurs** qui sont **données après le mot clé *in***.
 - Cette **variable est un compteur** car elle sert à **numéroter les itérations de la boucle**

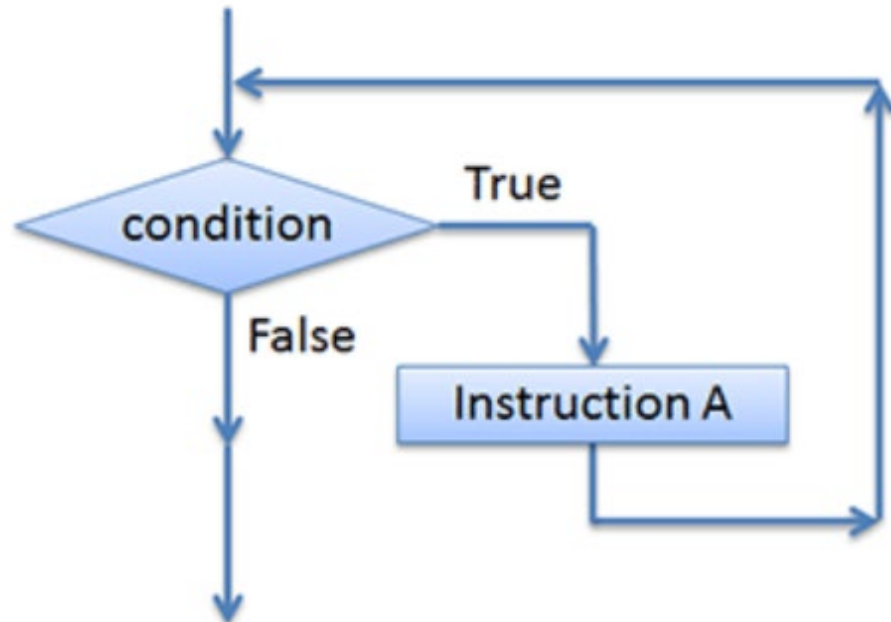
```
for i in [0, 1, 2, 3]:  
    print("i a pour valeur", i)
```

```
for lettre in "hello":  
    print (lettre)
```

L'instruction *while*

Syntaxe :

```
while condition:  
    Instruction A
```



L'instruction *while*

- L'instruction `while` (tant que) amorce une instruction composée

❖ Testez

```
>>> a = 0
>>> while (a < 7):           # (n'oubliez pas le double point !)
...     a = a + 1           # (n'oubliez pas l'indentation !)
...     print(a)
```

- Eviter les boucles sans fin !!!

```
>>> n = 3
>>> while n < 5:
...     print("hello !")
```

Construction d'une suite mathématique : Suite de Fibonacci

- Une suite de nombres dont chaque terme est égal à la somme des deux termes qui le précèdent

```
>>> a, b, c = 1, 1, 1
>>> while c < 11 :
...     print(b, end=" ")
...     a, b, c = b, a+b, c+1
```

Lorsque vous lancez l'exécution de ce programme, vous obtenez :

```
1 2 3 5 8 13 21 34 55 89
```

- L'argument **end=" "** signifie qu'on souhaite remplacer le saut de la ligne par une simple espace.

Variables	a	b	c
Valeurs initiales	1	1	1
Valeurs prises successivement, au cours des itérations	1	2	2
	2	3	3
	3	5	4
	5	8	5
	...	...	...
Expression de remplacement	b	a+b	c+1

Comment choisir entre boucle **for** et boucle **while**

En général, si on connaît avant de démarrer la boucle le nombre d'itérations à exécuter, on choisit une boucle **for**. Au contraire, si la décision d'arrêter la boucle ne peut se faire que par un test, on choisit une boucle **while**.

Note

Il est toujours possible de remplacer une boucle **for** par une boucle **while**.

Transformation d'une boucle **for** en un boucle **while**

```
i=0
for i in [0,1,2,3]:
    print ("i a pour valeur", i)
```

Le programme ci-dessus est équivalent à:

```
i = 0
while i < 4:
    print("i a pour valeur", i)
    i = i + 1
```

Un petit bonus

- $i = i + 1$
- $i += 1$

Exercices à faire

Faire un test pour savoir si a est compris dans l'intervalle allant de 2 à 8

Afficher les 10 premiers termes de la table de multiplication par 5.

Afficher les 10 premiers termes de la table de multiplication par 5, en signalant au passage (à l'aide d'un astérisque) ceux qui sont des multiples de 3.

- Exemple : 7 14 21* 28 35 42* 49

Afficher une suite de 10 nombres dont chaque terme soit égale au triple du terme précédent